

Design and Analysis of Algorithms

Subhasish Basak

July 31, 2026

Outline I

- 1 Course Overview
- 2 Fundamentals
- 3 Divide and Conquer

Course Description & Objectives

- Offers a **rigorous foundation** in algorithm design and analysis.
- Masters asymptotic notation, fundamental searching and sorting, and core problem-solving paradigms.
- Covers divide-and-conquer, greedy, and dynamic programming approaches.
- Explores advanced graph traversals, network flows, and string matching algorithms.
- Investigates computational complexity and NP-completeness, including practical strategies like approximation algorithms and heuristics.

Grading Policy

Evaluation will follow a **Relative Grading Policy**:

Assessment Component	Weightage
Assignments	20%
In-Semester Exam I	20%
In-Semester Exam II	20%
End-Semester Examination	40%

Table: Course Weightage Breakdown

Course Materials & References

- **Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein**, *Introduction to Algorithms*.
- **Jon Kleinberg and Eva Tardos**, *Algorithm Design*.
- **Robert Sedgewick and Kevin Wayne**, *Algorithms*.
- **Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman**, *The Design and Analysis of Computer Algorithms*.

How Do You Do It?

Think about how you solve these tasks instinctively in daily life:

- **The Contact List Search:** Looking up a friend's phone number. Do you scan every single entry from top to bottom, or do you open it near the middle?
- **Sorting Cards:** Arranging a shuffled hand of playing cards in numerical order. How do you pick up and slot each card into place?
- **Finding a Route:** Navigating from your room to the lecture hall. Do you randomly drive down every street, or do you use familiar landmarks and shortcuts?

Core Question: An algorithm is simply a formal, step-by-step recipe for the mental processes you already use to solve these puzzles!

Defining an Algorithm Formally

- **The Everyday View:** A precise sequence of instructions designed to accomplish a specific goal.
- **The Computer Science Definition:** A well-defined computational procedure that takes a specific set of values as **input** and produces a set of values as **output**.
- **Essential Requirements:**
 - **Clarity:** Every step is unambiguous and strictly defined.
 - **Finiteness:** It must successfully finish and stop running after a finite number of steps.
 - **Effectiveness:** The instructions must be basic enough to be carried out practically.

Notions of Problems and Algorithms

1. The Computational Problem (The "What")

- Defines the exact relationship between a valid **input** and the desired **output**.
- **Example:** The Sorting Problem.
 - *Input:* A sequence of n numbers.
 - *Output:* A reordering of those numbers into non-decreasing order.

2. Problem Instance vs. Algorithm (The "Data" vs. The "How")

- **Problem Instance:** One specific set of data fed into the problem (e.g., sorting the exact sequence $A = \langle 5, 2, 9, 1 \rangle$).
- **Algorithm:** The unambiguous, step-by-step procedure that guarantees the correct output for *any* valid problem instance.

Correctness and Computational Models

Algorithm Correctness

- An algorithm is **correct** if it successfully executes on every input instance and produces the correct output.
- A correct algorithm solves the given computational problem.

The RAM Model of Computation

- The **Random Access Machine (RAM)** model provides a theoretical framework for analyzing performance.
- Operations (arithmetic, data movement, control flow) take a constant amount of time.
- Allows us to count operations and measure efficiency independent of specific hardware.

Algorithms as a Technology

- **Why do algorithms matter?** Computing hardware is fast but not infinitely fast; memory is cheap but not free.
- **Efficiency Impact:** An efficient algorithm on a slower machine can often outperform an inefficient algorithm on a faster machine for large input sizes.
- **System Foundation:** Forms the core of computational tasks, enabling complex systems like network routing, cryptography, and data compression.

Standard Multiplication: By Hand

The Grade-School Method

- Let's multiply two 4-digit numbers: 1234×5678 .
- We multiply every digit of the top number by every digit of the bottom number, adding carries along the way.

1234				
x 5678				
9872	(8 × 1234:	4 mults,	3 carry adds)	
86380	(7 × 1234:	4 mults,	3 carry adds)	
740400	(6 × 1234:	4 mults,	3 carry adds)	
+6170000	(5 × 1234:	4 mults,	3 carry adds)	
7006652	(Summing rows:	≈ 16	column adds)	

Exact Step Count:

- 28** single-digit multiplications and additions (approx $2n^2$).
- ≈ **16** column additions to sum the final shifted numbers.
- For n digits, both multiplications and additions scale at $3n^2$.

Karatsuba's Method: The Concept

The Approach

- Instead of multiplying n digits all at once, we split the large numbers into two smaller halves.
- If we have two numbers x and y , we can split them at the middle (where $m = n/2$):
- $x = 10^m a + b$ (where a is the left half, b is the right half)
- $y = 10^m c + d$ (where c is the left half, d is the right half)
- Expanding the multiplication yields:

$$x \times y = 10^{2m}(ac) + 10^m(ad + bc) + bd$$

- To solve this directly, we need **4** separate multiplications: ac , ad , bc , and bd .
- Karatsuba realized we can isolate the middle term by subtracting ac and bd :

$$ad + bc = (a + b)(c + d) - ac - bd$$

- This reduces the required multiplications from **4** down to **3**.

Karatsuba's Method: The Setup

- Split the 4-digit numbers ($n = 4$) into 2-digit halves ($n/2 = 2$).
- $x = 1234 \implies a = 12, \quad b = 34$
- $y = 5678 \implies c = 56, \quad d = 78$

The Three Core Calculations

- 1 **Top halves:** $ac = 12 \times 56 = 672$
- 2 **Bottom halves:** $bd = 34 \times 78 = 2652$
- 3 **The Sums:**
 $(a + b) \times (c + d) = (12 + 34) \times (56 + 78) = 46 \times 134 = 6164$

Finding the Middle Term (No Extra Multiplication!)

- Middle = Step 3 – Step 1 – Step 2
- Middle = $6164 - 672 - 2652 = 2840$

Karatsuba's Method: Step Count

Analyzing the Recursion (Multiplications + Additions)

- Let $T(n)$ be the total operations. Adding or subtracting n -digit numbers takes steps proportional to n (we will call this $k \cdot n$).
- Standard:** 4 recursive multiplications + simple additions.

$$T_{std}(n) = 4T_{std}(n/2) + k_1 n$$

- Karatsuba:** 3 recursive multiplications + more complex additions.

$$T_{kar}(n) = 3T_{kar}(n/2) + k_2 n$$

The Trade-Off

- Karatsuba requires **more** additions at each step ($k_2 > k_1$) to compute $(a + b)$, $(c + d)$, and the subtractions.
- For $n = 4$: Standard does 16 multiplications + ≈ 28 adds. Karatsuba does 9 multiplications + ≈ 40 adds.

A Brief Digression: Recurrence Relations

Defining the Future Using the Past

- When an algorithm divides a problem into smaller subproblems, we describe its exact step count using a **recurrence relation**.
- It is an equation that defines a function in terms of its own values on smaller inputs.

The Classic Example: Fibonacci Sequence

- The Fibonacci sequence is naturally defined recursively: each number is the sum of the two preceding ones.
- **Recurrence:** $F_n = F_{n-1} + F_{n-2}$
- **Base Cases:** $F_0 = 0$, $F_1 = 1$
- Without base cases, the recursion would drop into negative numbers and never terminate.

Solving Recurrences: Mathematical Methods

Moving Beyond Guesswork

- To find a closed-form (exact non-recursive) formula for sequences like Fibonacci ($F_n = F_{n-1} + F_{n-2}$), we use formal algebraic tools.

Standard Algebraic Techniques

- **Characteristic Equations (Power Method):** We guess a solution of the form r^n . For Fibonacci, this creates the polynomial $r^2 - r - 1 = 0$. Solving for the roots gives the golden ratio and leads directly to Binet's Formula.
- **Substitution Method:** Here we guess the form of the solution and verify it with induction or some other tool.

Solving Recurrences: The Expansion Method

Unrolling the Recursion

- Instead of complex variable transformations, we can solve divide-and-conquer recurrences using the **expansion method** (or iteration).
- We repeatedly substitute the recurrence into itself until a clear mathematical pattern emerges.

The Karatsuba Recurrence

- Our equation: $T(n) = 3T\left(\frac{n}{2}\right) + cn$
- Let's assume a base case where $T(1) = c_0$ (a constant cost for a 1-digit multiplication).
- Let's see what happens when we "unroll" this step-by-step.

Unrolling Step-by-Step

Expanding Level by Level

- **Level 0 (Start):**

$$T(n) = 3T\left(\frac{n}{2}\right) + cn$$

- **Level 1 (Substitute $T(n/2)$):**

$$T(n) = 3 \left[3T\left(\frac{n}{4}\right) + c\left(\frac{n}{2}\right) \right] + cn = 3^2 T\left(\frac{n}{4}\right) + cn \left(1 + \frac{3}{2}\right)$$

- **Level 2 (Substitute $T(n/4)$):**

$$T(n) = 3^3 T\left(\frac{n}{8}\right) + cn \left(1 + \frac{3}{2} + \left(\frac{3}{2}\right)^2\right)$$

Generalizing and Reaching the Base Case

After k Expansions

- After k steps (where $k = \log_2 n$), the problem size shrinks down to our base case $T(1)$:

$$T(n) = 3^k T(1) + cn \sum_{i=0}^{k-1} \left(\frac{3}{2}\right)^i = 3^k T(1) + 2cn((1.5)^k - 1) \approx 3^k T(1)$$

Analyzing the Two Parts

- The First Term ($3^k T(1)$):** Grows **exponentially** with the depth of the recursion (k). Since $3^k = 3^{\log_2 n} = n^{\log_2 3} \approx n^{1.58}$.
- The Second Term (The Extra Additions):** At each level, the addition work scales linearly (cn), but because we sum a geometric series with ratio $\frac{3}{2}$, the total addition work scales exponentially as well but slower than the multiplication work.

Scaling Up: Why n^2 vs $n^{1.58}$ Matters

The True Power of Better Algorithms

- For 4-digit numbers, saving 7 multiplications seems trivial.
- But in modern cryptography (like RSA), computers multiply numbers with **thousands** of digits. Let's look at the exact multiplication steps required as n grows:

Digits (n)	Standard (n^2)	Karatsuba ($3^{\log_2 n}$)
4	16	9
16	256	81
64	4,096	729
1,024	$\approx 1,000,000$	59,049

Conclusion

- For a 1024-digit number, standard multiplication takes 1 million steps. Karatsuba takes less than 60,000—it is roughly **17 times faster**.
- *This* is why we study algorithms. We are now going to apply this exact mindset to searching.

Linear Search: Concept

The Computational Problem

- *Input:* A sequence of n numbers $A = \langle a_1, a_2, \dots, a_n \rangle$ and a search value v .
- *Output:* An index i such that $v = A[i]$, or NIL if v does not appear in A .

Description of the Algorithm

- Start at the first element of the sequence.
- Compare the current element to the target value v .
- If they match, stop and return the current position.
- If they do not match, move to the next element.
- If the end of the sequence is reached without a match, return a "not found" indicator.

Linear Search: Pseudocode

Pseudocode Representation

```
LINEAR-SEARCH(A, v)
1. for i = 1 to A.length
2.     if A[i] == v
3.         return i
4. return NIL
```

Why use Pseudocode?

- It strips away the strict syntax rules of specific programming languages.
- It focuses entirely on the logical procedure and control flow.
- It is universally understood regardless of whether a student codes in Python, C++, or Java.

Exact Computation Time (Worst-Case Analysis)

Let n be the length of array A . In the RAM model, we assign a constant cost c_k to each execution of a statement. Assume the worst-case scenario where v is not in A .

Statement	Cost	Times Executed
1. for $i = 1$ to $A.length$	c_1	$n + 1$
2. if $A[i] == v$	c_2	n
3. return i	c_3	0
4. return NIL	c_4	1

Total Running Time $T(n)$:

$$T(n) = c_1(n + 1) + c_2(n) + c_4(1)$$

$$T(n) = (c_1 + c_2)n + (c_1 + c_4)$$

Key Takeaway: The exact time depends on the specific hardware constants (c_k), but the runtime grows strictly linearly with the input size n .

From Exact Time to Asymptotic Growth

The Limits of Exact Analysis

- Our exact runtime was $T(n) = an + b$ (where $a = c_1 + c_2$ and $b = c_1 + c_4$).
- Finding exact hardware constants (c_k) for every line of code is tedious and restricts our analysis to a specific machine.

The Asymptotic Abstraction

- We care most about the **rate of growth** as the input size becomes extremely large ($n \rightarrow \infty$).
- For massive inputs, lower-order terms (like $+b$) and leading constants (like the multiplier a) lose their significance. The dominant term dictates performance.

Introducing Big- $\mathcal{O}$ Notation

- By dropping hardware-specific constants and lower-order terms, $T(n) = an + b$ abstracts to $T(n) = \mathcal{O}(n)$.
- We will discuss this in more detail later.

Can We Do Better Than $\mathcal{O}(n)$?

Revisiting the Contact List

- Linear Search checks every element one by one. In the worst case, finding a specific contact in a list of 10,000 people takes 10,000 operations.
- **Question for the Class:** When you look up a name in a physical dictionary or a printed phonebook, do you start at page 1 and read every single word?
- How do you actually find the target, and what specific property of the dictionary allows you to use that method?

Binary Search

The Prerequisite

- Unlike Linear Search, Binary Search strictly requires the input sequence A to be **sorted** (e.g., in non-decreasing order).

Description of the Algorithm

- **Step 1:** Identify the middle element of the current search space.
- **Step 2:** Compare the target value v with this middle element.
- **Step 3:** If they match, the search is successful. Return the middle index.
- **Step 4:** If v is less than the middle element, discard the right half. Repeat the search strictly on the left half.
- **Step 5:** If v is greater, discard the left half. Repeat the search strictly on the right half.

The Advantage

- Instead of reducing the problem size by 1 at each step, we reduce it by half. This dramatically minimizes the number of required operations.

Binary Search: Pseudocode

Iterative Pseudocode Representation

```
BINARY-SEARCH(A, v, low, high)
1. while low <= high
2.     mid = (low + high) / 2 // integer division
3.     if A[mid] == v
4.         return mid
5.     elseif A[mid] > v
6.         high = mid - 1
7.     else
8.         low = mid + 1
9. return NIL
```

From here-on we will skip the standard description and will stick to the pseudo-code.

Binary Search: Complexity Analysis

Analyzing the Loop

- Inside the `while` loop, we perform a constant number of basic operations (calculating the midpoint, one or two comparisons, and assigning a new boundary). This takes $\mathcal{O}(1)$ time.
- The runtime depends entirely on how many times the loop executes.

Halving the Search Space

- Initially, the search space is n .
- After 1 iteration: $n/2$ elements remain.
- After 2 iterations: $n/4$ elements remain.
- After k iterations: $n/2^k$ elements remain.

Binary Search: Asymptotic Time Complexity

Calculating the Worst-Case Runtime

- The worst-case scenario occurs when the search space is reduced to 1 element (or the target is not found).
- We equate the remaining elements to 1:

$$\frac{n}{2^k} = 1$$

- Multiplying both sides by 2^k yields:

$$n = 2^k$$

- Taking the base-2 logarithm of both sides gives the maximum number of iterations k :

$$k = \log_2 n$$

Final Complexity

- Since the loop runs at most $\log_2 n$ times and each iteration takes $\mathcal{O}(1)$ time, the worst-case time complexity is $\mathcal{O}(\log n)$.
- *Note:* The best-case complexity is $\mathcal{O}(1)$

Linear vs. Binary Search: The Sorting Trade-off

Performance Comparison

- **Linear Search:** $\mathcal{O}(n)$ time. Works on any unstructured array.
- **Binary Search:** $\mathcal{O}(\log n)$ time. Strictly requires a sorted array.

The Hidden Cost of Sorting

- The best general-purpose sorting algorithms (like Merge Sort or Heap Sort) take $\mathcal{O}(n \log n)$ time.
- Sorting an array and then performing a Binary Search takes $\mathcal{O}(n \log n) + \mathcal{O}(\log n)$ time.
- Asymptotically, this sorting cost is strictly worse than just doing a single Linear Search in $\mathcal{O}(n)$ time.

When is Binary Search Effective?

Single Query vs. Multiple Queries

- **Search Once:** If you only need to look up a value a single time, do not waste time sorting the data. Use **Linear Search**.
- **Search Many Times:** If you need to perform k searches on the same dataset, the cost analysis changes dramatically.

The Break-Even Point

- Cost of k Linear Searches: $k \cdot \mathcal{O}(n)$
- Cost of sorting once + k Binary Searches: $\mathcal{O}(n \log n) + k \cdot \mathcal{O}(\log n)$
- **Conclusion:** Binary Search is effective when k is large. The upfront computationally expensive cost of sorting is amortized across many extremely fast queries.

Why do we ignore constants?

Looking Back at Karatsuba

- Our unrolled equation yielded exact terms like $3^k T(1)$ and a geometric series multiplied by cn .
- For standard multiplication, the exact time might look like $T_{std}(n) = 5n^2 + 3n + 2$.
- For Karatsuba, it looks like $T_{kar}(n) = 12n^{1.58} + 8n$.

The Asymptotic Perspective

- Notice that Karatsuba has a higher leading constant ($12 > 5$). For very small n , standard multiplication might actually be faster!
- But as n grows toward infinity, the **dominant term** (n^2 vs $n^{1.58}$) completely dictates the running time.
- We need a mathematical notation that strips away hardware constants and lower-order terms to classify the pure *rate of growth*.

Big- $\mathcal{O}$ Notation: Formal Definition

The Mathematical Bound

- Let $f(n)$ and $g(n)$ be functions mapping non-negative integers to real numbers.
- We say $f(n) \in \mathcal{O}(g(n))$ if there exist positive constants $c > 0$ and $n_0 \geq 0$ such that:

$$0 \leq f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

What Do These Constants Mean?

- **c (The Multiplier):** We are allowed to scale the target function $g(n)$ by some constant factor to make it larger than $f(n)$.
- **n_0 (The Threshold):** The bound doesn't have to hold for tiny inputs. It only needs to hold after some specific input size n_0 .
- **Intuition:** Big- $\mathcal{O}$ provides an *asymptotic upper bound*. It guarantees the algorithm will scale no worse than $g(n)$.

Big- $\mathcal{O}$ Notation: Examples

Example 1: Dropping Lower-Order Terms

- Let $f(n) = 3n^2 + 5n + 10$.
- As n grows, $3n^2$ dwarfs $5n + 10$. We drop the lower terms and the constant 3.
- $f(n) = \mathcal{O}(n^2)$.

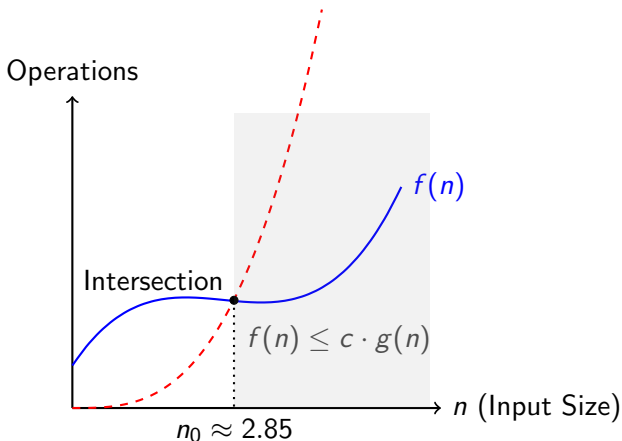
Example 2: Proving with the Formal Definition

- Prove that $2n + 5 = \mathcal{O}(n)$.
- We need $2n + 5 \leq c \cdot n$ for all $n \geq n_0$.
- If we choose $n_0 = 5$, then for all $n \geq 5$, $5 \leq n$.
- So, $2n + 5 \leq 2n + n = 3n$.
- By choosing $c = 3$ and $n_0 = 5$, the inequality holds perfectly.

Visualizing Big- $\mathcal{O}$: Ignoring the Noise

Graphical Proof: $\frac{1}{2}n^3 - 4n^2 + 10n + 5 = \mathcal{O}(n^3)$

$$c \cdot g(n) = 0.55n^3$$



Big- $\mathcal{O}$ Notation: Crucial Caveats

Warning 1: Constants Must Be Constant!

- The definition requires c and n_0 to be fixed, strictly independent of n .
- **A Common Trap:** Trying to prove $n^2 = \mathcal{O}(n)$ by writing $n^2 \leq c \cdot n$ and illegally choosing $c = n$.
- Because n grows toward infinity, c is no longer a constant. This invalidates the entire mathematical proof.

Warning 2: Upper Bounds Don't Have to be Tight

- Big- $\mathcal{O}$ only guarantees that an algorithm scales *no worse than* $g(n)$.
- Mathematically, lower powers of n are perfectly valid upper bounds of themselves. It is 100% correct to state that $5n = \mathcal{O}(n^3)$ or $2n = \mathcal{O}(2^n)$.
- **The Analogy:** If an algorithm takes 5 seconds, saying it is $\mathcal{O}(n^3)$ is like saying "it will finish in under a billion years." It is true, but practically useless.
- This is why we always seek the *tightest possible* upper bound.

Exercise: Determine the Complexity

Test Your Understanding

For each of the following exact time complexities, determine the tightest Big- $\mathcal{O}$ bound by identifying the dominant term.

- 1 $T(n) = 100n + 5000$
- 2 $T(n) = 4n^3 + 20n^2 + 10n$
- 3 $T(n) = n \log_2 n + 5n$
- 4 $T(n) = 2^n + n^{100}$

Discussion Question:

- Can we correctly say that $T(n) = 2n$ is $\mathcal{O}(n^3)$?

Big-Ω Notation: Formal Definition

The Mathematical Lower Bound

- Let $f(n)$ and $g(n)$ be functions mapping non-negative integers to real numbers.
- We say $f(n) \in \Omega(g(n))$ if there exist positive constants $c > 0$ and $n_0 \geq 0$ such that:

$$0 \leq c \cdot g(n) \leq f(n) \quad \text{for all } n \geq n_0$$

What Do These Constants Mean?

- **c (The Multiplier):** We scale our baseline function $g(n)$ by a constant factor to ensure it sits *below* $f(n)$.
- **n_0 (The Threshold):** Just like Big- $\mathcal{O}$, the bound only needs to hold for sufficiently large inputs.
- **Intuition:** Big- Ω provides an *asymptotic lower bound*. It guarantees the algorithm will take **at least** this much time. It represents the best-case or fundamental limits of a problem.

Big-Ω Notation: Examples

Example 1: Lower-Order Terms Still Don't Matter

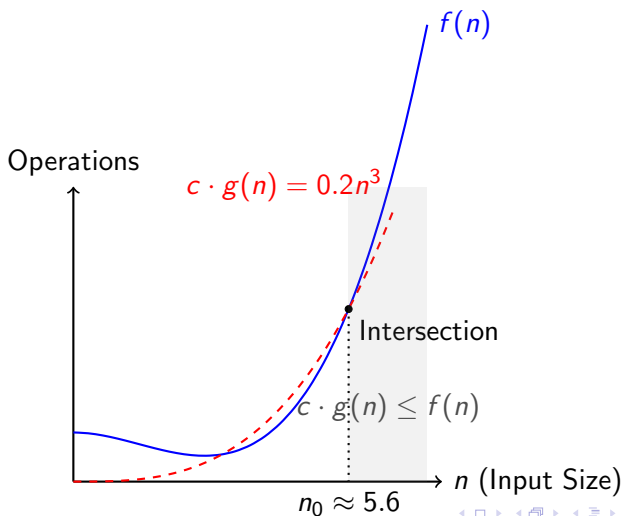
- Let $f(n) = 3n^2 - 5n$.
- Even though we are subtracting $5n$, as $n \rightarrow \infty$, the $3n^2$ term completely dominates.
- $f(n) = \Omega(n^2)$.

Example 2: Proving with the Formal Definition

- Prove that $5n^2 - 3n = \Omega(n^2)$.
- We need $c \cdot n^2 \leq 5n^2 - 3n$ for all $n \geq n_0$.
- Let's choose $n_0 = 1$. For $n \geq 1$, we know $3n \leq 3n^2$.
- Therefore, $5n^2 - 3n \geq 5n^2 - 3n^2 = 2n^2$.
- By choosing $c = 2$ and $n_0 = 1$, the inequality $2n^2 \leq 5n^2 - 3n$ holds perfectly.

Visualizing Big-Ω: Bounding from Below

Graphical Proof: $\frac{1}{2}n^3 - 2n^2 + 10 = \Omega(n^3)$



Big- Ω Notation: The "Loose Bound" Problem

Lower Bounds Don't Have to be Tight

- Just like Big- $\mathcal{O}$, Big- Ω only guarantees one side of the inequality. It says an algorithm scales *no better than* $g(n)$.
- Mathematically, 1 or n are perfectly valid lower bounds for n^3 . It is 100% correct to state that $n^3 = \Omega(n)$ or even $n^3 = \Omega(1)$.
- **The Analogy:** If an algorithm actually takes a million years to run (n^3), saying it is $\Omega(1)$ is like saying "it will take at least 1 second to finish."
- It is absolutely true, but it tells us nothing useful about the algorithm's actual performance limit.

Big- Θ Notation: Formal Definition

The Exact Asymptotic Bound

- We say $f(n) \in \Theta(g(n))$ if there exist positive constants $c_1 > 0$, $c_2 > 0$, and $n_0 \geq 0$ such that:

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{for all } n \geq n_0$$

What Does This Mean?

- Big- Θ is simply the intersection of Big- $\mathcal{O}$ and Big- Ω .
- If an algorithm is bounded from above ($\mathcal{O}$) and from below (Ω) by the **same** function $g(n)$, it is bounded tightly.
- **Intuition:** This is the gold standard. When we say an algorithm is $\Theta(n^2)$, we know its growth rate is *exactly* quadratic—no faster, no slower.

Big- Θ Notation: Examples

Example 1: The Tightness Guarantee

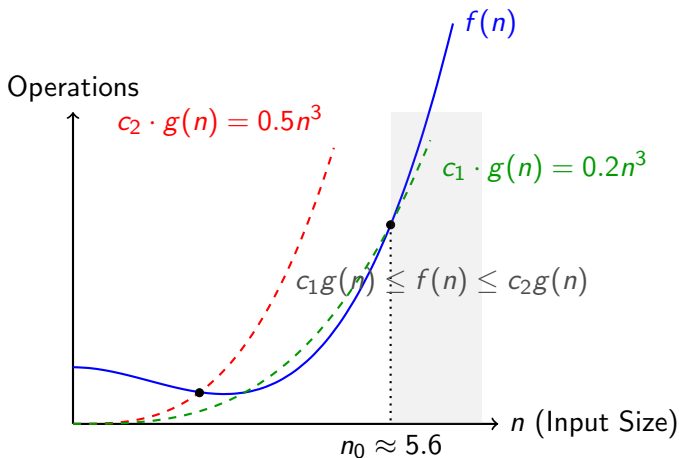
- Recall that $5n = \mathcal{O}(n^3)$ is mathematically true, but $5n \neq \Theta(n^3)$ because we cannot find a constant c_1 such that $c_1 \cdot n^3 \leq 5n$ as $n \rightarrow \infty$.
- The only function that bounds $5n$ from both sides asymptotically is a linear function. So, $5n = \Theta(n)$.

Example 2: Proving with the Formal Definition

- Prove that $\frac{1}{2}n^2 - 3n = \Theta(n^2)$.
- Upper Bound (c_2):** $\frac{1}{2}n^2 - 3n \leq \frac{1}{2}n^2$ is true for all $n \geq 0$. So $c_2 = 0.5$.
- Lower Bound (c_1):** We need $c_1 n^2 \leq \frac{1}{2}n^2 - 3n$.
- If we choose $c_1 = 0.25$, we get $0.25n^2 \leq \frac{1}{2}n^2 - 3n \implies 3n \leq 0.25n^2 \implies n \geq 12$.
- By choosing $n_0 = 12$, $c_1 = 0.25$, and $c_2 = 0.5$, the inequalities hold perfectly.

Visualizing Big-Θ:

Graphical Proof: $\frac{1}{2}n^3 - 2n^2 + 10 = \Theta(n^3)$



Summary of Asymptotic Notations

Notation	Analogy	Meaning
Big- $\mathcal{O}$	$\leq$ (Less than or equal)	Asymptotic Upper Bound
Big- Ω	$\geq$ (Greater than or equal)	Asymptotic Lower Bound
Big- Θ	$=$ (Equal to)	Exact Asymptotic Growth (Tight bound)
Little- o	$<$ (Strictly less than)	Strict Asymptotic Upper Bound

Special Mention: Little- o Notation

- While Big- $\mathcal{O}$ includes the possibility of a tight bound ($n^2 = \mathcal{O}(n^2)$ is true), Little- o means $f(n)$ grows **strictly slower** than $g(n)$.
- $n = o(n^2)$ is true, but $n^2 = o(n^2)$ is **false**.
- Limit Definition:** $f(n) = o(g(n))$ if and only if:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

Solving Recurrences: The Substitution Method

Guess and Prove

- The substitution method is a formal mathematical way to solve recurrences by guessing the answer and proving it using **mathematical induction**.
- **Step 1:** Guess the asymptotic upper bound.
- **Step 2:** Substitute the guess into the recurrence to prove it holds.

Example: $T(n) = 2T(n/2) + n$

- **Guess:** $T(n) = \mathcal{O}(n \log n)$. This means we must prove $T(n) \leq cn \log n$ for some constant $c > 0$.
- **Assume** the bound holds for smaller values like $n/2$:

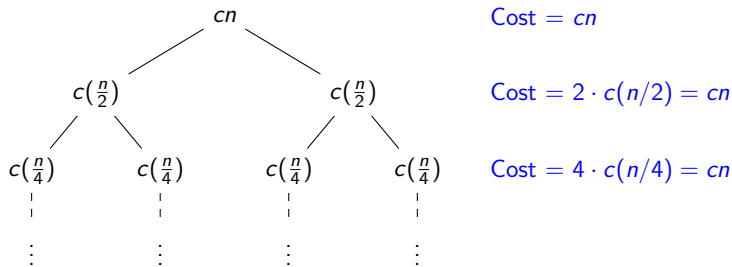
$$T(n/2) \leq c(n/2) \log(n/2)$$

- **Substitute** into the original recurrence:

$$\begin{aligned} T(n) &\leq 2(c(n/2) \log(n/2)) + n \\ &= cn(\log n - \log 2) + n \end{aligned}$$

Solving Recurrences: Recursion Tree Method

Visualizing the Work: $T(n) = 2T(n/2) + cn$



Summing the Tree

- At each level, the total non-recursive work is exactly cn .
- How many levels deep does the tree go? We divide by 2 until we hit a base case of size 1. The depth is $\log_2 n$.
- **Total Work** = (Cost per level) $\times$ (Number of levels) = $\Theta(n \log n)$.

The Master Theorem

- The Master Theorem provides a cookbook method for solving recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

where $a \geq 1$ (number of subproblems) and $b > 1$ (factor by which problem shrinks).

- We compare the root work $f(n)$ to $n^{\log_b a}$ (the work done at the leaves).
- **Case 1 (Leaves Dominate):** If $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, the leaves do the most work.

$$T(n) = \Theta(n^{\log_b a})$$

- **Case 2 (Balanced):** If $f(n) = \Theta(n^{\log_b a})$, work is evenly distributed across all levels of the tree.

$$T(n) = \Theta(n^{\log_b a} \log n)$$

The Master Theorem

Case 3 (Root Dominates)

- If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, the root *might* do the most work, **if** it also satisfies the regularity condition.

$$T(n) = \Theta(f(n))$$

The Regularity Condition

- For Case 3 to apply, $f(n)$ must not only be polynomially larger than $n^{\log_b a}$, but it must also satisfy:

$$af(n/b) \leq cf(n)$$

for some constant $c < 1$ and all sufficiently large n .

- **What does this mean?** The term $af(n/b)$ is the total work done at the *next* level down the tree. The condition $af(n/b) \leq cf(n)$ ensures that the work strictly decreases by a constant factor at every step down.
- Without this, $f(n)$ could fluctuate wildly, and the root wouldn't truly

The Master Theorem: Examples

Example 1: $T(n) = 9T(n/3) + n$

- $a = 9, b = 3, f(n) = n$.
- $n^{\log_3 9} = n^2$.
- Compare: $f(n) = n$ grows polynomially *slower* than n^2 .
- **Conclusion (Case 1):** $T(n) = \Theta(n^2)$.

Example 2: $T(n) = 2T(n/2) + n$

- $a = 2, b = 2, f(n) = n$.
- $n^{\log_2 2} = n^1 = n$.
- Compare: $f(n) = n$ grows at the *exact same rate* as the watershed.
- **Conclusion (Case 2):** Multiply by $\log n$. $T(n) = \Theta(n \log n)$.

Example 3: $T(n) = 3T(n/4) + n \log n$

- $a = 3, b = 4, f(n) = n \log n$.
- $n^{\log_4 3} \approx n^{0.79}$.
- Compare: $f(n) = n \log n$ grows polynomially *faster* than $n^{0.79}$.
- **Conclusion (Case 3):** $T(n) = \Theta(n \log n)$.

When the Master Theorem Fails

1. The Recurrence is the Wrong Shape

- The Master Theorem strictly requires $T(n) = aT(n/b) + f(n)$.
- **Subtraction instead of division:** $T(n) = T(n-1) + n$. The problem size doesn't shrink by a constant factor $b > 1$.
- **Variable number of subproblems:** $T(n) = nT(n/2) + n$. Here, $a = n$, but the theorem requires a to be a constant.

2. The Non-Polynomial Gap (The "No Man's Land")

- For Cases 1 and 3, $f(n)$ and the critical function $n^{\log_b a}$ must differ by a **polynomial factor** n^ϵ for some $\epsilon > 0$.
- **Example:** $T(n) = 2T(n/2) + n \log n$
- The critical function is $n^{\log_2 2} = n$.
- $f(n) = n \log n$ is strictly larger than n , so it looks like Case 3. However, $\frac{n \log n}{n} = \log n$, which grows slower than n^ϵ for any $\epsilon > 0$.
- **Result:** The standard Master Theorem completely fails here. You must use the recursion tree method, substitution method, or the Extended Master Theorem!

The Extended Master Theorem

- The Extended Master Theorem expands **Case 2** to handle cases where $f(n)$ differs from $n^{\log_b a}$ by a factor of $\log^k n$, rather than a polynomial factor (n^ϵ).
- If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some constant k , we use these rules:

Condition	Asymptotic Bound
$k > -1$	$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
$k = -1$	$T(n) = \Theta(n^{\log_b a} \log \log n)$
$k < -1$	$T(n) = \Theta(n^{\log_b a})$

Solving the Failed Example: $T(n) = 2T(n/2) + n \log n$

- $a = 2$, $b = 2$, so the critical function is $n^{\log_2 2} = n$.
- $f(n) = n \log^1 n$, which exactly matches the pattern with $k = 1$.
- Since $1 > -1$, we apply the first rule and add 1 to the logarithm's power:

$$T(n) = \Theta(n \log^{1+1} n) = \Theta(n \log^2 n)$$

Motivating Divide and Conquer: Computing Powers

The Problem

- Given a base a and an integer exponent $n \geq 0$, compute a^n .

The Naive Approach

- The most straightforward way is to repeatedly multiply a by itself n times.
- $a^n = a \times a \times a \times \cdots \times a$ (n times).

NAIVE-POWER(a , n)

```
1. result = 1
2. for i = 1 to n
3.     result = result * a
4. return result
```

Time Complexity: $\Theta(n)$. The number of multiplications scales strictly linearly with the exponent n . For a massive n , this becomes highly inefficient. Can we do better?

Computing Powers: Divide and Conquer

The Divide and Conquer Strategy

- Instead of multiplying by a one step at a time, we can halve the exponent at each step, solving a much smaller sub-problem.
- If n is even: $a^n = a^{n/2} \cdot a^{n/2} = (a^{n/2})^2$
- If n is odd: $a^n = a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a = (a^{(n-1)/2})^2 \cdot a$

The Recursive Algorithm (Exponentiation by Squaring)

DC-POWER(a , n)

1. if $n == 0$ return 1
2. $temp = DC-POWER(a, \text{floor}(n / 2))$
3. if n is even
4. return $temp * temp$
5. else
6. return $temp * temp * a$

Computing Powers: Complexity Analysis

Recurrence Relation

- Let $T(n)$ be the time (number of multiplications) to compute a^n .
- We solve exactly one sub-problem of half the size ($n/2$), and then perform either 1 or 2 additional constant-time multiplications.
- $T(n) = T(n/2) + \Theta(1)$

Solving the Recurrence

- By unwinding the recurrence (or using the Master Theorem):

$$T(n) = T(n/2) + c = T(n/4) + 2c = \dots = T(1) + c \log_2 n$$

- **Time Complexity:** $\Theta(\log n)$.

Why this matters:

- To compute 2^{1024} , the naive method requires 1024 multiplications.
- The divide-and-conquer method requires only about 10 multiplications! This illustrates the immense performance leap Divide and Conquer provides.

Standard Matrix Multiplication

The Problem

- Given two $n \times n$ matrices, A and B , compute their product $C = A \cdot B$.
- The standard mathematical definition states that each element C_{ij} is the dot product of the i -th row of A and the j -th column of B :

$$C_{ij} = \sum_{k=1}^n A_{ik} \cdot B_{kj}$$

The Standard Algorithm

SQUARE-MATRIX-MULTIPLY(A, B, n)

1. Let C be a new $n \times n$ matrix
2. for $i = 1$ to n
3. for $j = 1$ to n
4. $C[i, j] = 0$
5. for $k = 1$ to n
6. $C[i, j] = C[i, j] + A[i, k] * B[k, j]$
7. return C

Matrix Multiplication: A Divide and Conquer Approach

Block Matrix Multiplication

- We can partition each $n \times n$ matrix into four $n/2 \times n/2$ submatrices:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

- This yields four equations, for example: $C_{11} = A_{11}B_{11} + A_{12}B_{21}$.

Recurrence for Naive Divide and Conquer

- To compute all four quadrants of C , we need **8 recursive multiplications** of $n/2 \times n/2$ matrices, and **4 matrix additions**.
- Matrix addition takes $\Theta(n^2)$ time.
- The recurrence is: $T(n) = 8T(n/2) + \Theta(n^2)$.
- By the Master Theorem, since $\log_2 8 = 3$, the time complexity is $T(n) = \Theta(n^3)$.
- *Result:* No improvement over the standard nested loops!

Strassen's Algorithm: The Intuition

Breaking the $\Theta(n^3)$ Barrier

- In 1969, Volker Strassen discovered a way to compute the matrix product using only **7 recursive multiplications** instead of 8, at the cost of more additions.

The 7 Clever Products

- Strassen defined 7 intermediate matrices (P_1 through P_7) by combining parts of A and B using additions and subtractions before multiplying.
- For example: $P_1 = A_{11} \cdot (B_{12} - B_{22})$
- The four quadrants of C can then be reconstructed purely by adding and subtracting these P_k matrices.
- Example: $C_{11} = P_5 + P_4 - P_2 + P_6$
- Though algebraically complex, this completely eliminates one expensive $n/2 \times n/2$ matrix multiplication.

Strassen's Algorithm: The 7 Products (P_1 to P_7)

The 7 Sub-matrix Multiplications

- Strassen mathematically engineered 7 intermediate matrices.
- Each P_k requires exactly **one** multiplication of $n/2 \times n/2$ sub-matrices, plus some additions/subtractions:

$$P_1 = A_{11} \cdot (B_{12} - B_{22})$$

$$P_2 = (A_{11} + A_{12}) \cdot B_{22}$$

$$P_3 = (A_{21} + A_{22}) \cdot B_{11}$$

$$P_4 = A_{22} \cdot (B_{21} - B_{11})$$

$$P_5 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$P_6 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$

$$P_7 = (A_{11} - A_{21}) \cdot (B_{11} + B_{12})$$

Strassen's Algorithm: Reconstructing the Result

Combining the Products

- Once the 7 products are computed recursively, the four quadrants of the final matrix C are formed strictly through additions and subtractions ($\Theta(n^2)$ time):

$$C_{11} = P_5 + P_4 - P_2 + P_6$$

$$C_{12} = P_1 + P_2$$

$$C_{21} = P_3 + P_4$$

$$C_{22} = P_5 + P_1 - P_3 - P_7$$

Verification Example (C_{12})

- Let's expand C_{12} to prove it works:
- $P_1 + P_2 = A_{11}(B_{12} - B_{22}) + (A_{11} + A_{12})B_{22}$
- Expanding: $A_{11}B_{12} - A_{11}B_{22} + A_{11}B_{22} + A_{12}B_{22}$
- Canceling terms leaves: $A_{11}B_{12} + A_{12}B_{22}$

Strassen's Algorithm: Complexity Analysis

The New Recurrence Relation

- **Divide:** Partition matrices and compute 10 linear combinations of submatrices (Time: $\Theta(n^2)$).
- **Conquer:** Perform **7** recursive multiplications of $n/2 \times n/2$ matrices.
- **Combine:** Add and subtract the P_k matrices to reconstruct C (Time: $\Theta(n^2)$).
- The recurrence relation becomes:

$$T(n) = 7T(n/2) + \Theta(n^2)$$

Applying the Master Theorem

- Here, $a = 7$, $b = 2$, and $f(n) = \Theta(n^2)$.
- The critical exponent is $\log_b a = \log_2 7 \approx 2.807$.
- Since $f(n) = O(n^{\log_2 7 - \epsilon})$, the leaf-level work dominates the tree.
- **Final Time Complexity:** $T(n) = \Theta(n^{\log_2 7}) \approx O(n^{2.81})$.

Insertion Sort: The Intuition

Sorting a Hand of Cards

- Imagine you are dealt a hand of playing cards one by one. You hold the cards in your left hand and keep them sorted.
- When you receive a new card in your right hand, you compare it to the sorted cards from right to left.
- You shift the larger cards to the right to make room, and then *insert* the new card into its correct slot.

Algorithmic Strategy

- **In-place Sorting:** The array is conceptually divided into a "sorted" left portion and an "unsorted" right portion.
- Initially, the first element ($A[1]$) is trivially sorted by itself.
- The algorithm iterates through the remaining elements ($A[2]$ to $A[n]$), picking each one up and shifting the sorted elements until the correct insertion point is found.

Insertion Sort: Pseudocode

The Algorithm (1-Indexed)

```
INSERTION-SORT(A, n)
```

```
1. for j = 2 to n
```

```
2.     key = A[j]
```

```
3.     // Insert A[j] into the sorted sequence A[1..j-1]
```

```
4.     i = j - 1
```

```
5.     while i > 0 and A[i] > key
```

```
6.         A[i + 1] = A[i]      // Shift element to the right
```

```
7.         i = i - 1
```

```
8.     A[i + 1] = key          // Insert the key
```

- The for loop represents picking up each new card from the unsorted deck.
- The while loop handles the backward scanning and shifting of the already-sorted cards.

Insertion Sort: Worst-Case Analysis

The Worst-Case Scenario

- The array is strictly sorted in *reverse* order (e.g., $\langle 5, 4, 3, 2, 1 \rangle$).
- Every time we pick up a new key, it is smaller than **every** element in the sorted left portion.

Mathematical Summation

- For each j from 2 to n , the `while` loop must shift all $j - 1$ elements.
- The total number of shifts $T(n)$ is the sum of the first $n - 1$ integers:

$$T(n) = \sum_{j=2}^n (j - 1)$$

- Using the arithmetic series sum formula:

$$T(n) = 1 + 2 + 3 + \cdots + (n - 1) = \frac{n(n - 1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

Conclusion: Dropping constants and lower-order terms yields $T(n) = \Theta(n^2)$.

Selection Sort: The Intuition

Finding the Smallest Card

- Imagine all your cards are laid out face up on a table.
- You scan the entire group to find the absolute smallest card and swap it into the first position.
- Next, you scan the remaining cards to find the second smallest, and swap it into the second position, repeating this process until finished.

Algorithmic Strategy

- **In-place Sorting:** Like Insertion Sort, the array is conceptually divided into a "sorted" left portion and an "unsorted" right portion.
- The algorithm repeatedly *selects* the smallest element from the unsorted portion.
- It then swaps this minimum element with the first element of the unsorted portion.

Selection Sort: Pseudocode

The Algorithm

```
SELECTION-SORT(A, n)
1. for i = 1 to n - 1
2.     min_idx = i
3.     // Find the minimum element in A[i..n]
4.     for j = i + 1 to n
5.         if A[j] < A[min_idx]
6.             min_idx = j
7.     // Swap the found minimum with A[i]
8.     exchange A[i] with A[min_idx]
```

- The outer for loop advances the boundary of the sorted prefix.
- The inner for loop scans the remaining unsorted suffix to locate the minimum element.

Selection Sort: Time Complexity Analysis

Data Insensitivity

- Unlike Insertion Sort, Selection Sort must always scan the remaining array to verify the minimum, even if the array is already perfectly sorted.
- The best-case and worst-case scenarios take the exact same amount of time.
- For each i from 1 to $n - 1$, the inner loop makes exactly $n - i$ comparisons.
- The total number of comparisons $T(n)$ is:

$$T(n) = \sum_{i=1}^{n-1} (n - i)$$

- Expanding the series yields the same arithmetic progression:

$$T(n) = (n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n = \Theta(n^2)$$

Bubble Sort: The Intuition

Bubbling to the Top

- Imagine stepping *backward* through the array and comparing adjacent pairs of elements.
- If a pair is in the wrong order (right element is smaller), you swap them.
- By doing this repeatedly, the smallest (or "lightest") element is pushed to the very beginning of the array, like a bubble floating up to the surface.

Bubble Sort: Pseudocode

The Algorithm

BUBBLE-SORT(A, n)

```
1. for  $i = 1$  to  $n - 1$ 
2.     // Bubble smallest element to the left sorted bound
3.     for  $j = n$  downto  $i + 1$ 
4.         if  $A[j] < A[j - 1]$ 
5.             exchange  $A[j]$  with  $A[j - 1]$ 
```

- The outer for loop tracks the left boundary of the sorted portion.
- The inner for loop steps *backwards*, handling adjacent swapping until it hits the sorted prefix.

Bubble Sort: Analysis

- For each pass i from 1 to $n - 1$, the inner loop makes exactly $n - i$ comparisons.
- The total number of comparisons $T(n)$ is:

$$T(n) = \sum_{i=1}^{n-1} (n - i)$$

- Which sums up to our familiar arithmetic series:

$$T(n) = \frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

Conclusion: Dropping constants yields $T(n) = \Theta(n^2)$.

Loop Invariant Example: Bubble Sort

The Invariant for Left-Side Bubble Sort

- *Statement:* At the start of each iteration i of the outer for loop, the subarray $A[1 \dots i - 1]$ is sorted and contains the $i - 1$ smallest elements of the array.

Proving the Invariant

- **Initialization** ($i = 1$): The subarray is $A[1 \dots 0]$, which is empty. An empty array is trivially sorted and contains the 0 smallest elements.
- **Maintenance:** The inner loop compares adjacent elements stepping backwards down to $i + 1$. This safely "bubbles" the minimum element of the unsorted portion into position $A[i]$. The sorted prefix grows by one element and remains sorted.
- **Termination** ($i = n$): The sorted prefix is $A[1 \dots n - 1]$, containing the $n - 1$ smallest elements. Thus, $A[n]$ must be the largest element, meaning the entire array is sorted!

Beyond the Worst Case

Question

- We saw that Insertion, Selection, and Bubble Sort all share a worst-case time complexity of $\Theta(n^2)$.
- *Question:* Does this mean they perform identically in all situations? What happens in the **best case**?

The Best-Case Scenario (Already Sorted Array)

- **Selection Sort:** Still blindly scans the array to verify the minimum. Time complexity remains $\Theta(n^2)$.
- **Bubble Sort:** In the standard implementation, it still compares everything, yielding $\Theta(n^2)$ (*though an modified version with a swap flag drops to $\Theta(n)$*).
- **Insertion Sort:** Terminates its inner loop immediately. It only makes 1 comparison per item. Time complexity drops naturally to $\Theta(n)$.

Merge Sort: The Intuition

The Divide-and-Conquer Philosophy

- Quadratic sorts ($\Theta(n^2)$) slow down significantly for large datasets because elements only move one step or swap locally.
- **Merge Sort** adopts a **Divide-and-Conquer** strategy to break the problem into manageable sub-problems, solve them recursively, and combine the results.

The Three-Step Process

- **Divide:** Split the n -element sequence into two halves of $n/2$ elements each.
- **Conquer:** Recursively sort the two sub-sequences using Merge Sort.
- **Combine:** Merge the two sorted sub-sequences back together to produce the final sorted answer.

Merge Sort: The Core Operation (Merge)

How Combining Works

- The heart of Merge Sort is the $\text{MERGE}(A, p, q, r)$ procedure.
- Imagine you have two sorted piles of cards facing up. You look at the top card of each pile, pick the smaller one, and place it into a new combined pile.
- Because both input piles are already sorted, this linear scan allows us to combine them efficiently without nested loops.

Efficiency of Merging

- Merging two sorted sub-arrays totaling n elements takes $\Theta(n)$ time.
- It requires extra memory (out-of-place sorting) to construct the merged result, trading space complexity for speed.

Merge Sort: Pseudocode

The Recursive Algorithm

MERGE-SORT(A , p , r)

```
1. if  $p < r$ 
2.      $q = \text{floor}((p + r) / 2)$     // Find the midpoint
3.     MERGE-SORT( $A$ ,  $p$ ,  $q$ )        // Conquer: Sort left half
4.     MERGE-SORT( $A$ ,  $q + 1$ ,  $r$ )    // Conquer: Sort right half
5.     MERGE( $A$ ,  $p$ ,  $q$ ,  $r$ )        // Combine: Merge sorted halves
```

- **Base Case:** When $p = r$, the sub-array has a length of 1 and is trivially sorted.
- The heavy lifting happens in the MERGE step as the recursion unwinds.

Merge Sort: The MERGE Procedure

```
MERGE(A, p, q, r)
1.  n1 = q - p + 1
2.  n2 = r - q
3.  Let L[1..n1+1] and R[1..n2+1] be new arrays
4.  Copy A[p..q] into L[1..n1]
5.  Copy A[q+1..r] into R[1..n2]
6.  L[n1+1] = infinity      // Dummy values
7.  R[n2+1] = infinity      // Dummy values
8.  i = 1; j = 1
9.  for k = p to r
10.     if L[i] <= R[j]
11.         A[k] = L[i]
12.         i = i + 1
13.     else
14.         A[k] = R[j]
15.         j = j + 1
```

Merge Sort: Understanding MERGE

How the Algorithm Works

- **Setup:** We create two temporary arrays, L (Left) and R (Right), and copy our two sorted sub-arrays into them.
- **Dummy values (∞):** Placing an infinity at the end of L and R acts as a stopper. It prevents us from having to constantly check if we've emptied one of the arrays during the loop.
- **The Merge Loop (Lines 9-15):** We iterate through the original array segment $A[p \dots r]$. At each step, we compare the top elements of L and R , taking the smaller one and overwriting $A[k]$.

Cost Analysis

- **Time:** Every element is copied to L/R once, and written back to A once. For n elements, this takes exactly $\Theta(n)$ time.
- **Space:** We must allocate extra memory for the L and R arrays, which requires $\Theta(n)$ auxiliary space.

Merge Sort: Recurrence Analysis

Setting up the Recurrence Relation

- Let $T(n)$ be the time to sort an array of size n .
- **Divide:** Finding the midpoint takes constant time, $\Theta(1)$.
- **Conquer:** Solving two sub-problems of size $n/2$ takes $2T(n/2)$.
- **Combine:** The MERGE step takes $\Theta(n)$ time.

The Master Recurrence

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

Merge Sort: Time Complexity Conclusion

Why Merge Sort Wins

- Using the **Recursion Tree** method or the **Master Theorem**:
 - The tree has a height of $\log_2 n$ because we repeatedly divide the array in half.
 - At each of the $\log_2 n$ levels, the total work done for merging is proportional to n .

$$T(n) = \text{Work per level} \times \text{Number of levels} = \Theta(n \log n)$$

- According to the Master Theorem, the critical term $\log_b a = 1$ here. So the leaves and the root work are well balanced, which implies $T(n) = \Theta(n \log(n))$.

Final Verdict

- **Time Complexity:** $T(n) = \Theta(n \log n)$ across worst, average, and best cases.
- **Comparison:** Vastly outperforms Insertion, Selection, and Bubble Sort ($\Theta(n^2)$) for large inputs.